```
#Set A Q1/Q2
from google.colab import files

uploaded = files.upload()
```

```
<IPython.core.display.HTML object>
```

```
Saving customer_details.csv to customer_details.csv
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Get uploaded filename
filename = list(uploaded.keys())[0]

# Read CSV
df = pd.read_csv(filename)

print("Dataset loaded successfully!")
print("Shape:", df.shape)

# Display first 5 rows
df.head()
```

```
Dataset loaded successfully!
Shape: (20000, 4)
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 20000,\n  \"fields\":
[\n    {\n      \"column\": \"customer_id\",\n      \"properties\": {\
n        \"dtype\": \"number\",\n        \"std\": 8679504,\n
\"min\": 2093,\n        \"max\": 44625658,\n
\"num_unique_values\": 20000,\n        \"samples\": [\n
22284367,\n          9260847,\n          26311194\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"sex\",\n      \"properties\": {\n
\"dtype\": \"category\",\n        \"num_unique_values\": 4,\n
\"samples\": [\n          \"Female\",\n          \"UNKNOWN\",\n
\"Male\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"customer_age\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 604.3215886848897,\n        \"min\": -
34.0,\n        \"max\": 2022.0,\n        \"num_unique_values\": 93,\n
\"samples\": [\n          56.0,\n          32.0,\n          22.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"tenure\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\": 31,\n
\"min\": 4,\n        \"max\": 133,\n        \"num_unique_values\":
112,\n        \"samples\": [\n          59,\n          6,\n
88\n        ],\n        \"semantic_type\": \"\",\n
```

```
\"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe","variable_name":"df"}
```

```
# Dataset information
print("Dataset Information:")
df.info()
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20000 entries, 0 to 19999
Data columns (total 4 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   customer_id   20000 non-null  int64
 1   sex           20000 non-null  object
 2   customer_age  20000 non-null  float64
 3   tenure        20000 non-null  int64
dtypes: float64(1), int64(2), object(1)
memory usage: 625.1+ KB
```

```
# Statistical summary
df.describe(include='all')
```

```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 11,\n  \"fields\": [\n
{\n      \"column\": \"customer_id\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 14326964.60217146,\n
\"min\": 2093.0,\n        \"max\": 44625658.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n
17600399.01985,\n          15609125.0,\n          20000.0\n        ],\
n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"sex\",\n      \"properties\": {\
n        \"dtype\": \"category\",\n        \"num_unique_values\": 4,\n
\"samples\": [\n          4,\n          \"15322\",\n
\"20000\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"customer_age\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 6950.4331049871025,\n        \"min\": -
34.0,\n        \"max\": 20000.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          262.22255,\n          38.0,\n
20000.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"tenure\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 7054.536812231458,\n        \"min\": 4.0,\n        \"max\":
20000.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n
44.3968,\n          35.0,\n          20000.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe"}
```

```
# Count missing values
missing_values = df.isnull().sum()
```

```
print("Missing values in each column:")
print(missing_values)
```

```
Missing values in each column:
customer_id     0
sex             0
customer_age    0
tenure          0
dtype: int64
```

```
# Find numerical columns
numeric_columns = df.select_dtypes(include=np.number).columns

# Fill missing numerical values with median
for column in numeric_columns:
    df[column] = df[column].fillna(df[column].median())

print("Missing numerical values handled.")
```

```
Missing numerical values handled.
```

```
# Find categorical columns
categorical_columns = df.select_dtypes(include=['object',
'category']).columns

# Fill missing categorical values with mode
for column in categorical_columns:
    if df[column].isnull().sum() > 0:
        df[column] = df[column].fillna(df[column].mode()[0])

print("Missing categorical values handled.")
```

```
Missing categorical values handled.
```

```
print("Remaining missing values:")
print(df.isnull().sum())
```

```
Remaining missing values:
customer_id     0
sex             0
customer_age    0
tenure          0
dtype: int64
```

```
print("Duplicate rows before cleaning:", df.duplicated().sum())

df = df.drop_duplicates()

print("Duplicate rows after cleaning:", df.duplicated().sum())
```

```
Duplicate rows before cleaning: 0
Duplicate rows after cleaning: 0
```

```
numeric_columns = df.select_dtypes(include=np.number).columns

for column in numeric_columns:

    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)

    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = df[
        (df[column] < lower_bound) |
        (df[column] > upper_bound)
    ]

    print(f"{column}: {len(outliers)} outliers")
```

```
customer_id: 671 outliers
customer_age: 2102 outliers
tenure: 558 outliers
```

```
for column in numeric_columns:

    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)

    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Cap lower outliers
    df[column] = np.where(
        df[column] < lower_bound,
        lower_bound,
        df[column]
    )

    # Cap upper outliers
    df[column] = np.where(
        df[column] > upper_bound,
        upper_bound,
        df[column]
    )

print("Outliers handled using IQR capping.")
```

```
Outliers handled using IQR capping.
```

```
for column in numeric_columns:

    rolling_median = df[column].rolling(
        window=3,
        center=True,
        min_periods=1
    ).median()

    df[column] = df[column].fillna(rolling_median)

print("Noise handling completed.")
```

```
Noise handling completed.
```

```
print("Final Dataset Shape:")
print(df.shape)
```

```
Final Dataset Shape:
(20000, 4)
```

```
print("Missing Values:")
print(df.isnull().sum())
```

```
Missing Values:
customer_id     0
sex             0
customer_age    0
tenure          0
dtype: int64
```

```
print("Duplicate Rows:")
print(df.duplicated().sum())
```

```
Duplicate Rows:
509
```

```
df.head()
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 20000,\n  \"fields\":
[\n    {\n      \"column\": \"customer_id\",\n      \"properties\": {\
n        \"dtype\": \"number\",\n        \"std\": 8404612.40776335,\n
\"min\": 2093.0,\n        \"max\": 37890380.5,\n
\"num_unique_values\": 19330,\n        \"samples\": [\n
9405011.0,\n          28871773.0,\n          29518607.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"sex\",\n      \"properties\": {\n
\"dtype\": \"category\",\n        \"num_unique_values\": 4,\n
\"samples\": [\n          \"Female\",\n          \"UNKNOWN\",\n
\"Male\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
```

```
\"customer_age\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 74.65319674620152,\n        \"min\": -
34.0,\n        \"max\": 264.0,\n        \"num_unique_values\": 93,\n
\"samples\": [\n          56.0,\n          32.0,\n          22.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"tenure\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\":
31.270829593714453,\n        \"min\": 4.0,\n        \"max\": 118.5,\n
\"num_unique_values\": 107,\n        \"samples\": [\n          22.0,\n
66.0,\n          88.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe","variable_name":"df"}
```

```
#Set B Q1
!pip install -q pyspark

from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("DataCleaning") \
    .master("local[*]") \
    .getOrCreate()

print("Apache Spark started successfully!")
```

```
Apache Spark started successfully!
```

```
filename = list(uploaded.keys())[0]

df = spark.read.csv(
    filename,
    header=True,
    inferSchema=True
)

print("Dataset loaded successfully!")
print("Number of rows:", df.count())
print("Number of columns:", len(df.columns))

df.show(5)
```

```
Dataset loaded successfully!
Number of rows: 20000
Number of columns: 4
+-----------+----+------------+------+
|customer_id| sex|customer_age|tenure|
+-----------+----+------------+------+
|    9798859|Male|        44.0|    93|
|   11413563|Male|        36.0|    65|
|     818195|Male|        35.0|   129|
|   12049009|Male|        33.0|    58|
```

```
|   10083045|Male|        42.0|    88|
+-----------+----+------------+------+
only showing top 5 rows
```

```
df.printSchema()
```

```
root
 |-- customer_id: integer (nullable = true)
 |-- sex: string (nullable = true)
 |-- customer_age: double (nullable = true)
 |-- tenure: integer (nullable = true)
```

```
from pyspark.sql.functions import col, when, count, trim

# ============================================
# CHECK MISSING VALUES - FIXED VERSION
# ============================================

missing_expr = []

for field in df.schema.fields:
    column = field.name

    if field.dataType.simpleString() == "string":
        # For string columns: NULL or empty/whitespace
        expr = count(
            when(
                col(column).isNull() | (trim(col(column)) == ""),
                1
            )
        ).alias(column)

    else:
        # For numeric/date/etc. columns: only NULL
        expr = count(
            when(col(column).isNull(), 1)
        ).alias(column)

    missing_expr.append(expr)

missing_values = df.select(missing_expr)

missing_values.show()
```

```
+-----------+---+------------+------+
|customer_id|sex|customer_age|tenure|
+-----------+---+------------+------+
|          0|  0|           0|     0|
+-----------+---+------------+------+
```

```
print("Rows before removing duplicates:", df.count())

df = df.dropDuplicates()

print("Rows after removing duplicates:", df.count())
```

```
Rows before removing duplicates: 20000
Rows after removing duplicates: 20000
```

```
from pyspark.sql.types import NumericType

numeric_columns = [
    field.name
    for field in df.schema.fields
    if isinstance(field.dataType, NumericType)
]

print("Numerical columns:")
print(numeric_columns)
```

```
Numerical columns:
['customer_id', 'customer_age', 'tenure']
```

```
for column in numeric_columns:
    median_value = df.approxQuantile(
        column,
        [0.5],
        0.01
    )[0]

    df = df.fillna({
        column: median_value
    })

print("Missing numerical values handled.")
```

```
Missing numerical values handled.
```

```
from pyspark.sql.functions import count, desc

categorical_columns = [
    field.name
    for field in df.schema.fields
    if field.dataType.simpleString() == "string"
]

for column in categorical_columns:

    mode_row = (
        df.groupBy(column)
        .count()
```

```
        .orderBy(desc("count"))
        .first()
    )

    if mode_row is not None and mode_row[0] is not None:
        mode_value = mode_row[0]

        df = df.fillna({
            column: mode_value
        })

print("Missing categorical values handled.")
```

```
Missing categorical values handled.
```

```
from pyspark.sql.functions import col

for column in numeric_columns:

    quantiles = df.approxQuantile(
        column,
        [0.25, 0.75],
        0.01
    )

    Q1 = quantiles[0]
    Q3 = quantiles[1]

    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    print("\nColumn:", column)
    print("Q1:", Q1)
    print("Q3:", Q3)
    print("IQR:", IQR)
    print("Lower Bound:", lower_bound)
    print("Upper Bound:", upper_bound)

    outlier_count = df.filter(
        (col(column) < lower_bound) |
        (col(column) > upper_bound)
    ).count()

    print("Number of outliers:", outlier_count)
```

```
Column: customer_id
Q1: 11751467.0
Q3: 22058337.0
```

```
IQR: 10306870.0
Lower Bound: -3708838.0
Upper Bound: 37518642.0
Number of outliers: 716

Column: customer_age
Q1: 29.0
Q3: 123.0
IQR: 94.0
Lower Bound: -112.0
Upper Bound: 264.0
Number of outliers: 2102

Column: tenure
Q1: 20.0
Q3: 59.0
IQR: 39.0
Lower Bound: -38.5
Upper Bound: 117.5
Number of outliers: 558
```

```python
from pyspark.sql.functions import when

for column in numeric_columns:

    quantiles = df.approxQuantile(
        column,
        [0.25, 0.75],
        0.01
    )

    Q1 = quantiles[0]
    Q3 = quantiles[1]

    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    df = df.withColumn(
        column,
        when(col(column) < lower_bound, lower_bound)
        .when(col(column) > upper_bound, upper_bound)
        .otherwise(col(column))
    )

print("Outliers handled using IQR capping.")
```

```
Outliers handled using IQR capping.
```

```
from pyspark.sql.functions import floor, min, max

for column in numeric_columns:

    stats = df.select(
        min(column).alias("min"),
        max(column).alias("max")
    ).collect()[0]

    min_value = stats["min"]
    max_value = stats["max"]

    if min_value is not None and max_value is not None:
        bin_width = (max_value - min_value) / 10

        if bin_width > 0:
            df = df.withColumn(
                column + "_bin",
                floor(
                    (col(column) - min_value) / bin_width
                )
            )

print("Noise handling using binning completed.")
```

```
Noise handling using binning completed.
```

```
print("Final number of rows:", df.count())
print("Final number of columns:", len(df.columns))

df.show(10)
```

```
Final number of rows: 20000
Final number of columns: 7
+-----------+------+------------+------+--------------
+----------------+----------+
|customer_id|   sex|customer_age|tenure|customer_id_bin|
customer_age_bin|tenure_bin|
+-----------+------+------------+------+--------------
+----------------+----------+
|1.0095813E7|  Male|        31.0|  88.0|              2|
2|         7|
|   829829.0|Female|        53.0| 117.5|              0|
2|        10|
|  8984101.0|  Male|        35.0|  99.0|              2|
2|         8|
|  8751981.0|  Male|       123.0| 101.0|              2|
5|         8|
|  8267763.0|  Male|        13.0| 104.0|              2|
1|         8|
|  9660928.0|  Male|        31.0|  95.0|              2|
```

```
2|         8|
|  9927071.0|  Male|        43.0|  91.0|              2|
2|         7|
|  9761955.0|  Male|        48.0|  94.0|              2|
2|         7|
|1.0059553E7|  Male|        34.0|  88.0|              2|
2|         7|
|1.1319525E7|Female|        31.0|  67.0|              3|
2|         5|
+-----------+------+------------+------+---------------
+----------------+----------+
only showing top 10 rows
```

```
#Set B-2
# ============================================
# SPLIT DATA INTO TRAINING AND TESTING SETS
# ============================================

# 80% Training, 20% Testing
train_df, test_df = df.randomSplit(
    [0.8, 0.2],
    seed=42
)

# Count records
total = df.count()
training = train_df.count()
testing = test_df.count()

# Display results
print("=" * 50)
print("DATASET SPLIT RESULTS")
print("=" * 50)

print("Total records      :", total)
print("Training records   :", training)
print("Testing records    :", testing)

print("Training percentage:",
      round(training / total * 100, 2), "%")

print("Testing percentage :",
      round(testing / total * 100, 2), "%")

# Display samples
print("\nTraining Dataset:")
train_df.show(5)

print("\nTesting Dataset:")
test_df.show(5)
```

```
==================================================
DATASET SPLIT RESULTS
==================================================
Total records      : 20000
Training records   : 16052
Testing records    : 3948
Training percentage: 80.26 %
Testing percentage : 19.74 %

Training Dataset:
+-----------+----+------------+------+--------------+----------------
+----------+
|customer_id| sex|customer_age|tenure|customer_id_bin|
customer_age_bin|tenure_bin|
+-----------+----+------------+------+--------------+----------------
+----------+
|     2093.0|Male|        47.0| 117.5|               0|
2|        10|
|    12817.0|Male|        40.0| 117.5|               0|
2|        10|
|    15155.0|Male|        42.0| 117.5|               0|
2|        10|
|    23205.0|Male|        41.0| 117.5|               0|
2|        10|
|    25012.0|Male|        48.0| 117.5|               0|
2|        10|
+-----------+----+------------+------+--------------+----------------
+----------+
only showing top 5 rows

Testing Dataset:
+-----------+------+------------+------+--------------
+----------------+----------+
|customer_id|   sex|customer_age|tenure|customer_id_bin|
customer_age_bin|tenure_bin|
+-----------+------+------------+------+--------------
+----------------+----------+
|    14309.0|Female|        37.0| 117.5|               0|
2|        10|
|    27083.0|Female|        34.0| 117.5|               0|
2|        10|
|    28749.0|  Male|        36.0| 117.5|               0|
2|        10|
|    32515.0|  Male|        38.0| 117.5|               0|
2|        10|
|    46753.0|  Male|        35.0| 117.5|               0|
2|        10|
+-----------+------+------------+------+--------------
```

```
+----------------+----------+
only showing top 5 rows
```